

Cloud vs Edge Pipeline Benchmark

Technical explanation of the Docker Compose benchmark, real TLS/mTLS communication, simulated security overhead, Prometheus metrics, and Grafana visualization.

```
Project: cloud-edge-pipeline-benchmark
Start: docker compose up --build
Run benchmark: docker compose run --rm benchmark
```

1. Purpose of the Project

This project is a local, reproducible benchmark environment for comparing cloud and edge processing pipelines. It shows how latency, synchronization cost, logical bandwidth usage, and security overhead change when the same workload is executed in different deployment models.

Core comparison: in the cloud pipeline, raw data is sent to the cloud and processed there. In the edge pipeline, data is processed near the source and only a reduced result is synchronized to the cloud.

2. High-Level Architecture

Docker Compose starts multiple services on a shared Docker network. Plain HTTP services and TLS/mTLS services are kept separate so their performance can be compared directly.

```
Plain cloud pipeline
Client -> cloud-api HTTP -> simulated network -> preprocessing -> inference -> storage

Plain edge pipeline
Client -> edge-api HTTP -> simulated local network -> preprocessing -> inference -> sync reduced result to cloud

TLS edge pipeline
Client -> HTTPS/mTLS -> edge-api-tls -> preprocessing -> inference -> HTTPS/mTLS sync to cloud-api-tls
```

3. Docker Compose Services

Service	Role	Port
certgen	Generates local CA, server certificates, and client certificate for TLS/mTLS.	N/A
cloud-api	Plain HTTP cloud processing service.	8000
edge-api	Plain HTTP edge processing service with CPU/RAM limits.	8001
cloud-api-tls	HTTPS/mTLS cloud processing service.	8443
edge-api-tls	HTTPS/mTLS edge service that syncs results to cloud-api-tls.	8444
benchmark	Python runner that calls every scenario and writes CSV/JSON output.	N/A
prometheus	Scrapes realtime metrics from the API services.	9090
grafana	Visualizes Prometheus metrics through a provisioned dashboard.	3000

4. How Startup Works

```
docker compose up --build
```

- 1 Builds the Python images for the API, benchmark runner, and certificate generator.
- 2 Runs certgen, which writes certificates into certs/.
- 3 Starts the plain HTTP cloud and edge APIs.

- 4 Starts the HTTPS/mTLS cloud and edge APIs after certificates exist.
- 5 Starts Prometheus and configures it to scrape both HTTP and HTTPS/mTLS endpoints.
- 6 Starts Grafana and provisions the Prometheus data source plus the dashboard.

Cloud API plain: http://localhost:8000/docs
Edge API plain: http://localhost:8001/docs
Cloud API TLS/mTLS: https://localhost:8443/docs
Edge API TLS/mTLS: https://localhost:8444/docs
Prometheus: http://localhost:9090
Grafana: http://localhost:3000

Note: the TLS endpoints use a local self-signed CA and require a client certificate. The benchmark and Prometheus are configured automatically. A browser may reject the TLS pages unless the local CA and client certificate are imported.

5. API Behavior

The API service is implemented with FastAPI in `services/pipeline_api/app/main.py`. The same image is reused for cloud and edge; behavior changes through environment variables such as `ROLE`, `NETWORK_UPLINK_MS`, `PREPROCESS_MS`, and `INFERENCE_MS`.

```
POST /process

{
  "input_id": "sample-001",
  "data_size_kb": 512,
  "complexity": 1.0,
  "security_profile": "none | simulated | tls"
}
```

Cloud mode

- 1 Apply simulated network uplink delay.
- 2 Optionally apply simulated security stages.
- 3 Run synthetic preprocessing.
- 4 Run synthetic inference.
- 5 Run simulated storage.
- 6 Return timing data to the client.

Edge mode

- 1 Apply smaller simulated local network delay.
- 2 Optionally apply simulated security stages.
- 3 Run synthetic preprocessing on the constrained edge container.
- 4 Run synthetic inference on the constrained edge container.
- 5 Send a reduced result to the cloud through `/sync`.
- 6 Return timing data to the client.

6. Benchmark Scenarios

The benchmark runner is implemented in `services/benchmark/benchmark.py`. Run it with:

```
docker compose run --rm benchmark
```

Scenario	Meaning	Transport
cloud	Raw data is sent to cloud and processed there.	HTTP
edge	Data is processed at the edge; a reduced result is synced to cloud	HTTP
cloud_simulated	Cloud pipeline with configurable simulated security overhead.	HTTP
edge_simulated	Edge pipeline with configurable simulated security overhead.	HTTP
cloud_tls	Cloud pipeline over real HTTPS/mTLS.	HTTPS/mTLS
edge_tls	Edge pipeline over real HTTPS/mTLS, including TLS sync to cloud	HTTPS/mTLS

```
RUNS: 30
DATA_SIZE_KB: 512
COMPLEXITY: 1.0
```

7. Security Model

Simulated security

The *_simulated_secure scenarios add explicit synthetic stages inside the API timer. This is useful for explaining the theoretical performance cost of communication security mechanisms.

- security_tls_handshake: simulated TLS or mTLS handshake cost.
- security_hmac_auth: simulated token/HMAC verification cost.
- security_encrypt_decrypt: simulated payload encryption/decryption cost, proportional to payload size.
- security_replay_check: simulated nonce/timestamp replay-protection cost.

Real TLS/mTLS

The *_tls scenarios use real HTTPS. Uvicorn is started with SSL settings, and each TLS service uses a certificate generated by certgen. The benchmark client verifies the local CA and presents a client certificate. This models mutual TLS: the client validates the server, and the server requires a trusted client certificate.

Real TLS overhead is not recorded as security_ms, because it happens before the request reaches the FastAPI handler. It appears mainly in client_service_delta_ms, the difference between client-observed end-to-end time and API-internal processing time.

8. Output Files and Interpretation

results/results.csv
results/summary.json

Column	Meaning
pipeline	Scenario name, such as cloud, edge_tls, or cloud_simulated_secure.
client_total_ms	Total latency measured by the benchmark client.
service_total_ms	Latency measured inside the API service.
client_service_delta_ms	Client service timing. For TLS, includes handshake, certificate verification, connection setup, HTTP overhead.
security_ms	Simulated security overhead. Non-zero only in simulated secure scenarios.
network_ms	Configured synthetic network delay.
preprocess_ms	Measured synthetic preprocessing time.
inference_ms	Measured synthetic inference time.
sync_ms	For edge pipelines, time spent synchronizing the reduced result to cloud.
payload_sent_kb	Logical input payload size sent by the client.
payload_synced_kb	Logical payload sent to cloud. Raw input for cloud, reduced result for edge.

9. Prometheus and Grafana

Prometheus does not run the benchmark. It periodically scrapes metrics exposed by each API service at /metrics. Grafana uses Prometheus as a data source to visualize these metrics.

API services expose:

```
pipeline_requests_total  
pipeline_total_latency_ms  
pipeline_stage_latency_ms  
pipeline_payload_size_kb  
pipeline_in_flight_requests
```

Prometheus scrapes:

```
http://cloud-api:8000/metrics  
http://edge-api:8000/metrics  
https://cloud-api-tls:8443/metrics  
https://edge-api-tls:8443/metrics
```

For TLS targets, Prometheus is configured with the local CA and client certificate. Monitoring itself is therefore performed over authenticated TLS.

10. Reading Typical Results

- The cloud pipeline often has lower compute time because the simulated cloud service has faster preprocessing and inference parameters.
- The edge pipeline has higher local compute time because the edge container is constrained with cpus: "1.0" and mem_limit: 512m.
- The edge advantage is lower logical cloud bandwidth: it synchronizes only the reduced result instead of the raw input.
- The simulated secure scenarios show a configurable overhead in security_ms.
- The real TLS scenarios show transport overhead mainly through client_service_delta_ms and, for edge, through higher sync_ms because edge-to-cloud sync also uses HTTPS/mTLS.

11. Useful Commands

```
# Start everything and build images
docker compose up --build

# Start in the background
docker compose up -d --build

# Run the benchmark once
docker compose run --rm benchmark

# Check service status
docker compose ps

# Watch API logs
docker compose logs -f cloud-api edge-api cloud-api-tls edge-api-tls

# Stop everything
docker compose down

# Regenerate certificates
docker compose run --rm -e FORCE_REGENERATE_CERTS=1 certgen
docker compose up -d --build
```

12. Limitations and Extensions

This environment is useful for controlled benchmarking and explanation, but it is still an emulation. It does not fully model thermal throttling, real sensor I/O, wireless interference, GPU/NPU acceleration, or production-grade certificate lifecycle management.

- Replace synthetic preprocessing/inference with a real ML model.
- Run the edge API on a Raspberry Pi, Jetson, or other real edge device.
- Add network emulation with tc netem or Containernet.
- Add concurrent users or streams to stress throughput and queueing behavior.
- Add real authentication tokens, signed payloads, and nonce persistence.
- Export Grafana panels for inclusion in a thesis or report.

13. Repository Notes

Generated certificates, private keys, and local benchmark outputs are intentionally ignored by Git:

```
certs/*.cert
certs/*.key
results/*.csv
results/*.json
```

This prevents accidental publication of private key material or machine-specific benchmark artifacts. The certificates can always be regenerated through the certgen service.